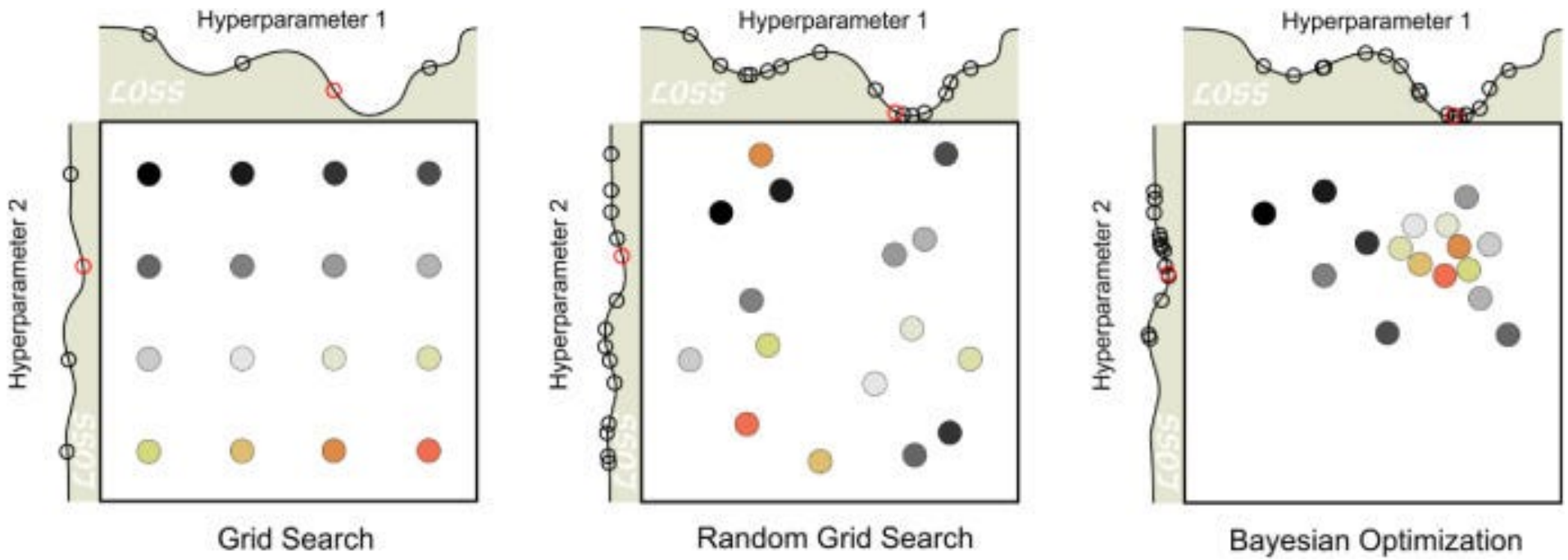
AutoML, Autoresearch, MLOps +@

26.4.7

서민교

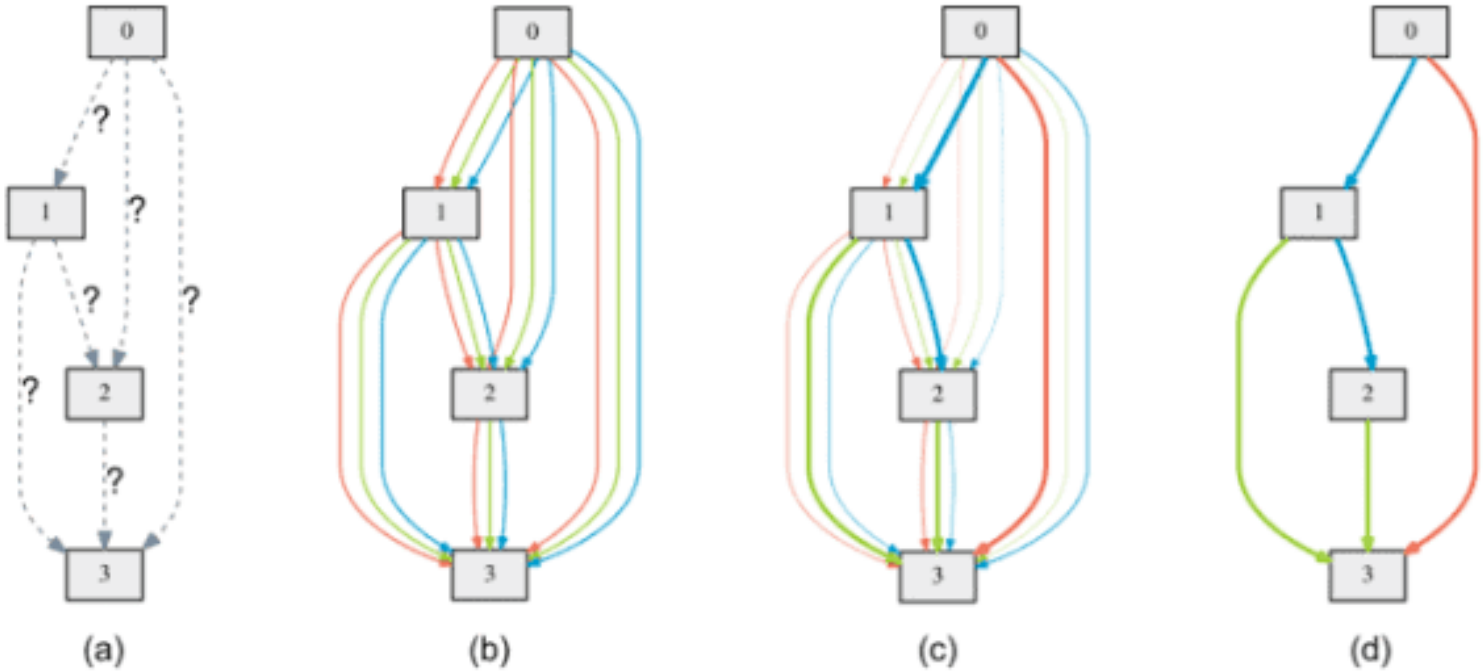
1. AutoML이란 무엇인가?

- 모델 개발 탐색 일부 자동화
- 대표 대상: `model selection`, `hyperparameter tuning`, `pipeline search`
- 핵심: 비교적 주어진 `search space` 안 최적 설정 탐색



2. `Neural Architecture Search`는 AutoML의 확장이다

- parameter 대신 architecture 탐색
- AutoML의 `더 넓은 search space` 확장선
- 그래도 중심은 여전히 모델/파이프라인 후보 탐색



`hyperparameter search → pipeline search → architecture search`

3. `Autoresearch`는 어떻게 등장했고 무엇이 다른가?

- karpathy/autoresearch: 작은 training setup 위 `read → edit → run → keep-or-revert` loop 제시
- 연구 workflow 일부를 agent가 직접 수행하며, 설정 탐색을 넘어 `code`, `module`, `experiment` 자체 수정
- AutoML의 `fixed search space` 바깥으로 확장
- 이후 RD-Agent, AI-Scientist, GPT Researcher 등으로 빠르게 확장

4. Agent 작업 흐름

- 코드 읽기, baseline 파악
- 작은 가설 하나 선택
- 학습 코드나 설정 수정
- 짧은 실험 실행, metric 확인
- 나쁘면 revert, 의미 있으면 keep
- 핵심: `edit 한 번`이 아니라 `짧은 실험 loop의 누적`

```
Question → Read → Edit → Run → Analyze → Next experiment
```

5. AutoML vs. Autoresearch

항목	AutoML	Autoresearch
탐색 대상	config, pipeline, architecture	hypothesis, code, module, experiment
핵심 질문	어떤 설정이 가장 좋은가	다음에 어떤 실험을 해야 하는가
edit 단위	parameter / architecture	code / module / pipeline / experiment
평가 방식	objective 중심	objective + reasoning + iteration
위험	비효율적 탐색	incoherent search, metric hacking
필요한 인프라	experiment infra	experiment + memory + harness

6. 사용례와 확장

사용례

- 문헌 조사 / deep research: GPT Researcher
- 코드 수정 + 실험 반복: karpathy/autoresearch, RD-Agent
- end-to-end 연구 자동화: AI-Scientist

확장

- benchmark / evaluation: MLE-bench, MLAgentBench, MLR-Bench
- plugin / skill 생태계: awesome-autoresearch, Awesome Auto Research Tools
- memory, reusable modules, hardware fork

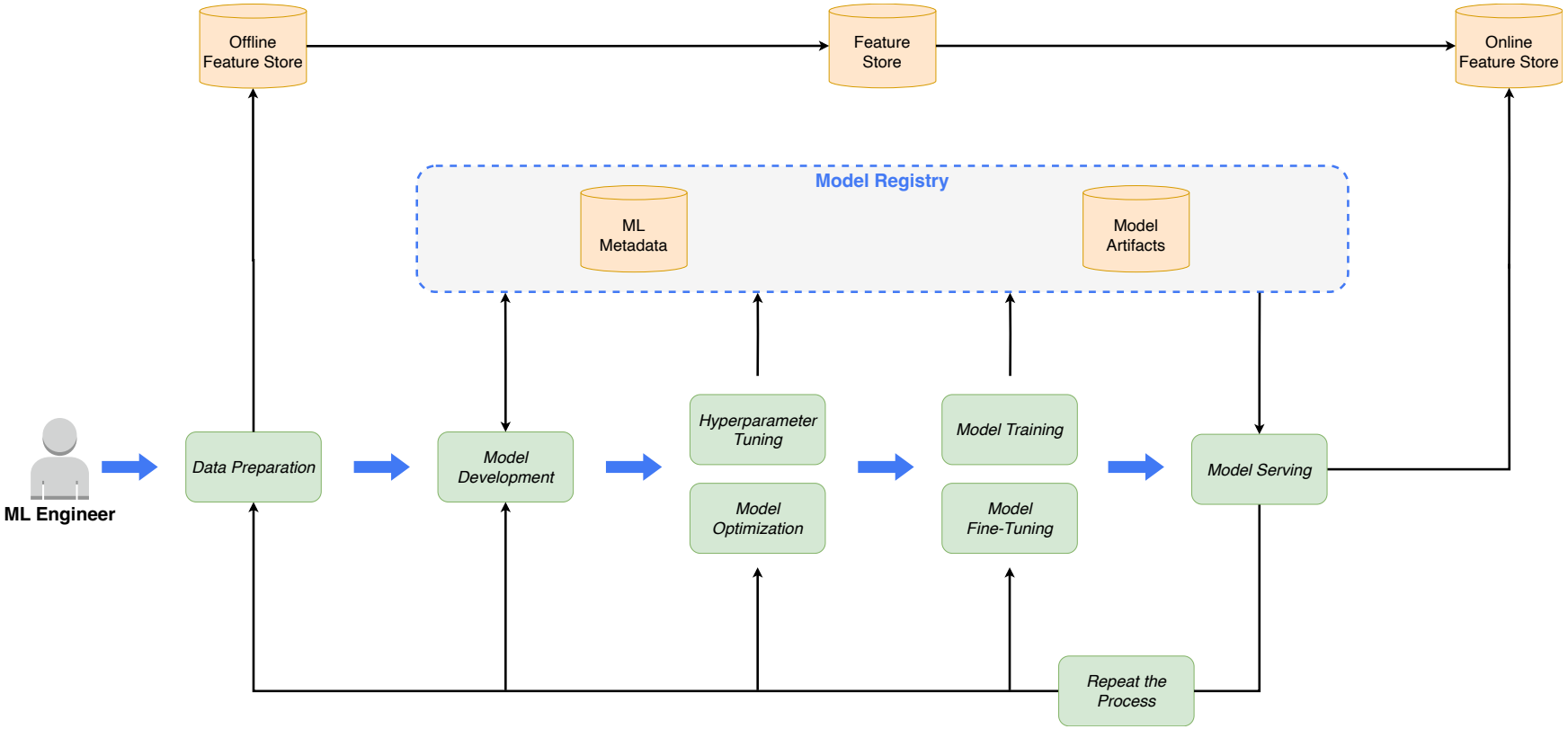
7. 체계적인 실험 관리의 필요

- 공통 문제: `많은 run` 비교와 누적
- 필수 요소: `tracking`, `lineage`, `orchestration`
- agent edit가 들어오면 `artifact`, `promotion`, `monitoring`, `cost control` 중요도 상승
- 결국 운영 문제

8. AutoML과 Autoresearch가 공통으로 요구하는 MLOps 요소

요소	AutoML에서의 역할	Autoresearch에서의 역할
tracking	sweep 비교	hypothesis / code edit history 비교
orchestration	search job 실행	agent + eval job 실행
registry / lineage	best model 승격	experiment / prompt / code provenance 보존
monitoring / cost	retrain trigger, SLO	budget, drift, unsafe promotion guardrail

9. MLOps는 모델 개발, 관리, 배포 파이프라인을 유지 관리하는 작업이다



- Autoresearch loop는 이 큰 ML lifecycle 안의 일부
- 실제 시스템: `data`, `experiment`, `model registry`, `deployment`, `monitoring`
- 핵심 역할: `지속 운영`, `추적`, `승격`, `유지관리`

10. Autoresearch의 단점

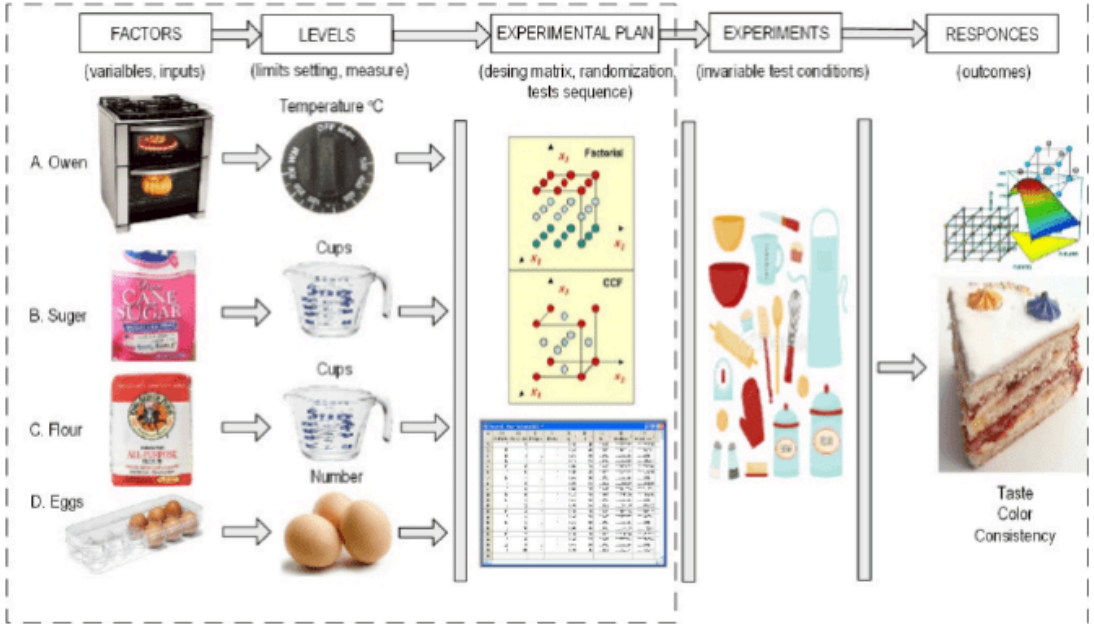
- 실험이 즉흥적으로 이어지기 쉬움
- 왜 이 실험을 했는지 attribution이 약함
- 큰 수정, 작은 튜닝, 검증 실험이 섞이기 쉬움
- robustness, replication, interaction 확인이 뒤로 밀림
- 잘 정리된 random search로 퇴화할 위험

11. 어떤 Harness가 필요한가

- 무엇을 먼저 볼지 정하는 우선순위
- 어떤 조합을 함께 볼지 정하는 규율
- 탐색 단계와 검증 단계 분리
- 작은 수정과 큰 수정을 다르게 다루는 운영 규칙
- 실패도 정보로 남기는 구조
- 다음 라운드를 설계하는 순차 실험 체계

12. Design of Experiments(DoE)란 무엇인가

- 여러 요인을 한 번에 바꿔 보며 effect를 읽는 실험 설계
- 한 번의 최고점보다 `요인`, `상호작용`, `안정성` 파악에 강점
- 핵심 질문: 무엇을 바꿨고, 무엇이 실제로 영향을 줬는가



13. DoE에서 빌려오는 개념

- `screening` : 중요한 요인부터 좁히기
- `factorial thinking` : interaction 보기
- `sequential design` : 라운드별 정교화
- `robust design` : 평균이 아니라 안정성까지 확인
- `mixture / allocation` : 예산과 비율 배분

14. DoE-guided 운영과 비교한 Agents

Agent	운영 방식	특징
`01 Sequential`	Vanilla autoresearch	최신 기록에 맞춰 작은 변경을 순차 적용
`02 Simple DoE`	DoE-guided screening	factor와 level을 두고 screening 중심 비교
`03 Advanced DoE Tic-Tac-To`	DoE-guided staged program	staged DoE + 실험 타입 예산 분배

- `Tic` : 둘 이상 모듈 동시 변경
- `Tac` : 한 모듈 교체
- `To` : 같은 구조 안 수치 조정
- 운영 규칙: `Tic:Tac:To = 1:2:4`

15. 실험 설정

Benchmark	데이터 / 제약	실제 사용 model	현재 시작점
`cifar10_real`	`max_samples=4000`	`mlp`	세 agent 모두 `mlp_anchor` control 에서 시작
`twenty_newsgroups_real`	`max_samples=8000`, `max_features=2000`, `ngram_max=2`, `min_df=2`	`mlp`	같은 loop로 benchmark만 전환

실행 조건

- agent별 isolated root를 따로 만들어 context leakage 없이 독립 실행
- 같은 start control에서 출발해 validation-only `200` runs 누적

MLP에서 바꿀 수 있는 요소

- preprocessing: `normalization`, `projection`, `projection_dim`
- model structure: `hidden_dims`
- optimization: `activation`, `solver`, `learning_rate_init`, `batch_size`
- regularization / norm: `normalization_layer`, `weight_decay`

16. 결과: 성능, 효율, 안정성

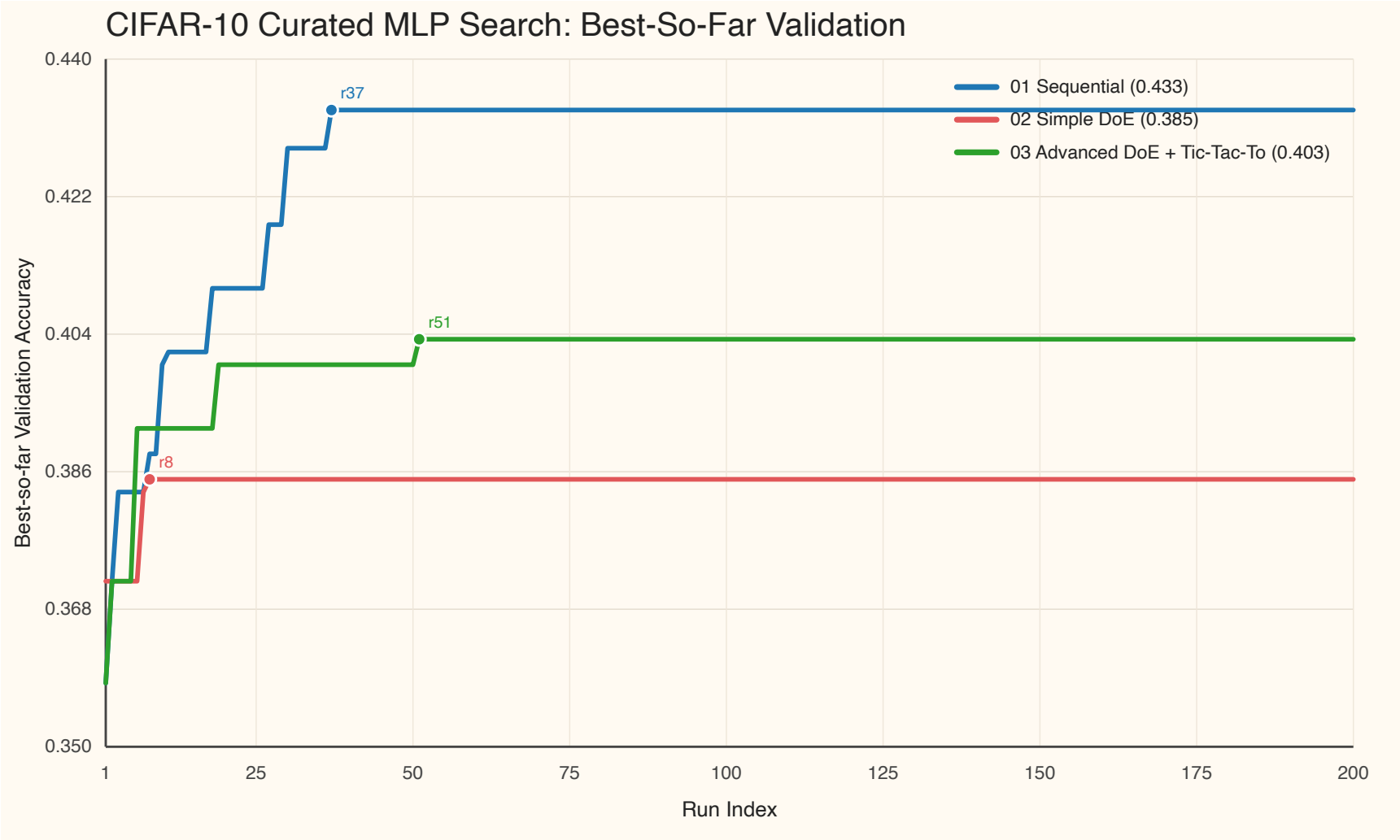
조건: `cifar10_real` / `mlp` / curated `10` knobs / validation only

Agent	Best Val	Run of Best	Gain vs Run 1	Mean Best-so-Far	Incumbent Updates
`01 Sequential`	`0.4333`	`37`	`+0.0750`	`0.4282`	`10`
`02 Simple DoE`	`0.3850`	`8`	`+0.0133`	`0.3846`	`3`
`03 Advanced DoE + Tic-Tac-To`	`0.4033`	`51`	`+0.0450`	`0.4012`	`5`

대표 config

- `01 Sequential`: `maxabs + svd(32) + [64,32] + relu + adam + batchnorm + wd=0.001 + lr=0.0005 + bs=32`
- `02 Simple DoE`: `standard + no projection + [64,32] + relu + adam + no internal norm + wd=0.0005 + lr=0.0003 + bs=128`
- `03 Advanced DoE + Tic-Tac-To`: `maxabs + svd(32) + [64,32] + relu + adam + no internal norm + wd=0.001 + lr=0.0005 + bs=64`

17. 결과: 탐색 궤적



- `01 Sequential` : `run 37` 에 최고점 도달 후 긴 plateau 유지
- `02 Simple DoE` : `run 8` 에 빠르게 best를 찾았지만 ceiling 상승은 약함

18. 히스토리에서 읽히는 결론

- `Sequential` 은 좁은 surface에서 가장 강했다.
- `Simple DoE` 는 early screening은 빨랐지만 ceiling을 많이 못 올렸다.
- `Advanced DoE + Tic-Tac-To` 는 중후반 탐색 건강도가 가장 좋았다.
- 작은 factor space에선 strong local refinement가 더 효율적일 수 있다.
- screening-only보다 staged refinement와 budget allocation이 더 중요했다.

19. 히스토리와 피드백에서 추출한 튜닝 지식

`01 Sequential`

- 강한 basin: `maxabs + svd(32) + [64,32] + relu + adam`
- `batchnorm`, `batch_size=32`, `lr=0.0005`, `wd=0.001` 쪽이 강했다.
- `sgd` 전환은 반복적으로 약했다.

`02 Simple DoE`

- 강한 basin: `standard + no projection + [64,32] + relu + adam + batch_size=128`
- 초반엔 `normalization`, `learning_rate_init`, `batch_size` screening이 효율적이었다.
- 이후 late-stage refinement policy가 따로 필요했다.

20. 히스토리와 피드백에서 추출한 튜닝 지식

`03 Advanced DoE + Tic-Tac-To`

- 강한 basin: `maxabs + svd(32) + [64,32] + relu + adam + batch_size=64`
- `lr=0.0005`, `wd=0.001` 근처가 반복적으로 살아남았다.
- `32x32`, `tanh`, `std+pca32` branch는 자주 약했다.
- `screening → interaction → local refinement`와 `Tic:Tac:To ≈ 1:2:4`가 long-horizon search에 유효했다.

메타 지식

- `history.md`와 `feedback.md`는 다음 benchmark에 재사용할 `configuration prior`와 `search policy prior` 원천이다.
- DOE 계열은 `hypothesis / factors / levels / conclusion` 구조가 남아 지식 추출이 쉽다.

21. 한계

- 문헌 조사나 가설 생성 자체를 대체하진 못함
- 완전히 open-ended한 구조 탐색은 factorization이 어려움
- classical DoE는 코드 중심 search space에 바로 맞지 않을 수 있음
- large-scale 학습에서는 replication 비용이 큼
- 평가 함수가 빈약하면 DoE도 잘못된 목표를 최적화할 수 있음

22. References

구분	예시
curated landscape	awesome-autoresearch, Awesome Auto Research Tools
end-to-end systems	karpathy/autoresearch, RD-Agent, AI-Scientist
deep research	GPT Researcher
evaluation	MLE-bench, MLAgentBench, MLR-Bench
visuals	AutoML image, NAS image, Kubeflow model registry lifecycle image